Gate 1: entrega y verificación de la miniaplicación (Bloque 11) / Repaso de la rúbrica del Gate 1 y las tres oportunidades

✓ **Hecho:** Ver

Repaso de la rúbrica del Gate 1 y las tres oportunidades

Con la **Unidad U1.5** cerrada, terminaste de construir cada pieza de tu miniaplicación: el entorno, el módulo, el manejo de errores, la arquitectura documentada, y los **prompts** con su salida estructurada.

El **Bloque 11** no te pide una pieza nueva de código: es el punto donde todo eso se evalúa junto, formalmente, en lo que este programa llama **Gate 1**.

Antes del ensayo de verificación de la siguiente lección y de la entrega que cierra el módulo, esta lección repasa con precisión qué se evalúa, cómo se aprueba, y qué pasa si tu primera entrega no alcanza.

🎯 Qué es un Gate, y en qué se diferencia de una actividad Core

Ya entregaste varias actividades marcadas como "Core" a lo largo de este módulo, y cada una se evaluó por sí sola.

> 🏁 **Un Gate es distinto**: es un hito obligatorio de cierre de módulo, que no pregunta si completaste una lección puntual, sino algo más amplio: ¿tienes las condiciones técnicas mínimas para seguir avanzando hacia el desarrollo con **IA**?

✅ Aprobar el **Gate 1** certifica esos fundamentos operativos y es lo que habilita tu paso al Módulo 2.

*Por eso el **Gate 1** reúne evidencia de todo el módulo, no solo del último bloque: tu repositorio, tu entorno, tu módulo **Python**, tu manejo de errores, tu arquitectura, tu decisión de modelo y tu banco de prompts*

*tu módulo **Python**, tu manejo de errores, tu arquitectura, tu decisión de modelo y tu banco de prompts entran todos en esta misma evaluación.*

📊 Las seis dimensiones de la rúbrica

La evaluación del **Gate 1** se reparte en seis dimensiones, cada una con un peso distinto sobre el resultado final:

Dimensión evaluada	Peso
Configuración del entorno y repositorio	20 %
Python aplicado y estructuración del código	20 %
Consumo funcional de API	25 %
Prompt y procesamiento de respuesta	15 %
Seguridad y credenciales	10 %
Documentación y versionamiento	10 %

⚠️ Los criterios que no se promedian

El promedio ponderado de esas seis dimensiones no es, por sí solo, suficiente para aprobar. Hay cuatro condiciones que el programa trata como no compensables:



Que tu **API** funcione.



Que tu entorno sea reproducible.



Que tus credenciales no queden expuestas.



Que tu código sea ejecutable.

Si cualquiera de estas cuatro falla, el Gate queda pendiente, aunque el promedio matemático alcance el umbral.

> 💡 **La razón es simple**: estas cuatro condiciones no son "una dimensión más" de calidad, son la base sin la cual el resto de la evaluación no tiene sentido.

Una aplicación que expone su **API key** en el repositorio no es **"un poco menos segura"**: es un riesgo real, sin importar qué tan bien esté escrito el resto del código.



🎯 Cómo se aprueba: el umbral y las tres oportunidades

Para aprobar el Gate 1 necesitas dos cosas a la vez:

- ✓ Un mínimo de 80 % de eficacia en el promedio ponderado de la rúbrica.
- ✓ Que ninguno de los cuatro criterios no compensables esté fallando.

Si no llegas a esa combinación en tu primera entrega, **no pierdes el módulo**: el programa contempla tres oportunidades de aprobación para cada Gate, y cada una cumple una función distinta.

Instancia	Qué evalúa
Oportunidad 1	Entrega regular
Oportunidad 2	Corrección sobre el feedback y las brechas identificadas
Oportunidad 3	Recuperación formal, enfocada solo en los criterios no logrados

Las tres oportunidades no significan **"entregar tres veces desde cero"**: si en tu primera entrega ya demostraste satisfactoriamente algunas dimensiones (por ejemplo, tu entorno y tu repositorio están perfectos), la segunda oportunidad se enfoca en corregir lo que falló, no en rehacer lo que ya funcionaba.

✅ Checklist de autochequeo, antes del ensayo

Antes de pasar a la lección siguiente, revisa tu propio proyecto contra los cuatro criterios no compensables. No hace falta el ensayo completo todavía (eso es exactamente el contenido de la próxima lección): alcanza con estas cuatro verificaciones rápidas, cada una con una forma concreta de confirmarla.

> ✓ **Código ejecutable sin errores**. Corre tu módulo de punta a punta ahora mismo, contra la API real. Si termina sin ninguna excepción sin capturar en pantalla, este punto está cubierto.

> ✓ .**gitignore activo, credenciales protegidas**. Corre "git status": tu archivo ".env" no debería aparecer como archivo nuevo ni modificado. Para confirmarlo de forma más estricta, corre "git ls-files | grep .env"; si el único resultado es tu ".env.example", estás bien. Si aparece tu ".env" real, ve al recuadro de abajo antes de seguir.

> ✓ **Entorno reproducible**. Esto se prueba a fondo en la próxima lección, con el ensayo completo. Como adelanto rápido, confirma que tu "requirements.txt" existe y está versionado ("git ls-files | grep requirements.txt"), y que no instalaste nada "a mano" que falte ahí.

requirements.txt"), y que no instalaste nada "a mano" que falte ahí.

✓ **Consumo exitoso de la API**. Si el punto 1 ya corrió sin errores y viste una respuesta real del modelo en tu consola (no un error 401, 429 o de conexión), este punto también está cubierto.

Si ya se filtró una credencial

Si tu archivo **".env"** o tu API key real ya quedaron subidos a Git (aunque sea en un commit antiguo), no alcanza con borrarlos y hacer un commit nuevo: siguen visibles en el historial. El primer paso, siempre, es **revocar o regenerar esa credencial** (en el panel de tu proveedor de modelo): una vez revocada, ya no sirve para nada, aunque siga visible en el historial, y eso suele ser suficiente para resolver el riesgo real.

Si necesitas limpiar el historial: por ejemplo, porque el repositorio es público, la herramienta que se recomienda hoy es **"git filter-repo"** (no **"git filter-branch"** ni herramientas de terceros ya discontinuadas), seguido de un **"push --force"** y de avisar a cualquier persona que tenga clonado tu repositorio.

Para evitar que vuelva a ocurrir: revisa también que tu **".gitignore"** incluya **".env"** desde ahora.

Siguiente lección

Bloque 11 · Verificación en vivo

Con la rúbrica y el sistema de oportunidades ya claros, en la siguiente lección vas a ensayar la parte más literal de esta evaluación: simular exactamente lo que hace quien evalúa tu **Gate**, clonando, instalando y ejecutando tu proyecto desde cero, sin ninguna ayuda tuya.

Última modificación: jueves, 24 de septiembre de 2026, 17:14